

FMN050 Numerical Analysis for E3

Solutions for Homework assignment 4

1. Matrix norm interpretation

1a) Let $P = (x, y)^T$ be any point at the unit circle in R^2 , so we have

$$\begin{aligned} |x| + |y| &= 1 && \text{for 1-norm} \\ x^2 + y^2 &= 1 && \text{for 2-norm} \\ \max(|x|, |y|) &= 1 && \text{for max-norm} \end{aligned}$$

We can use the following M-file to plot the pictures.

```
x = -1:0.001:1;
n = size(x,2);
figure
hold
axis equal
axis([-1.5 1.5 -1.5 1.5])

%% for 1-norm
plot(x, 1-abs(x), 'b', x, -(1-abs(x)), 'b');

%% for 2-norm
plot(x, sqrt(1-x.^2), 'g', x, -sqrt(1-x.^2), 'g');

%% for inf-norm
plot(x, 1, 'b', x, -1, 'b', -1, x, 'b', 1, x, 'b')

A = [sqrt(2)+1 1; 0 sqrt(2)];
figure
hold
axis equal
axis([-4 4 -4 4])

%%% for 1-norm
for i = 1:n
    y1(:,i) = A*[x(i);1-abs(x(i))];
    y2(:,i) = A*[x(i);-(1-abs(x(i)))];
end plot( y1(1,:),y1(2,:), 'b', y2(1,:),y2(2,:), 'b' );
```

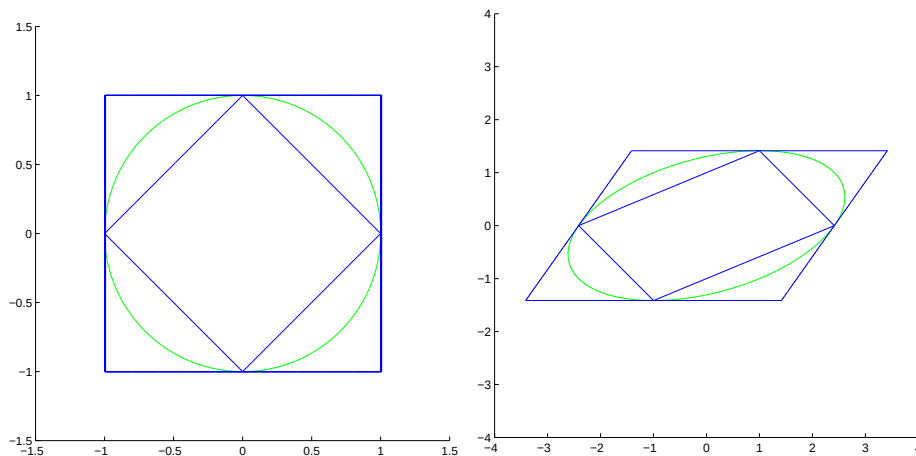
```

%% for 2-norm
for i = 1:n
    y1(:,i) = A*[x(i);sqrt(1-x(i)^2)];
    y2(:,i) = A*[x(i);-sqrt(1-x(i)^2)];
end plot( y1(1,:),y1(2,:), 'g', y2(1,:),y2(2,:), 'g' );

%% for inf-norm
for i = 1:n
    y1(:,i) = A*[x(i);1];
    y2(:,i) = A*[x(i);-1];
    y3(:,i) = A*[1;x(i)];
    y4(:,i) = A*[-1;x(i)];
end plot(y1(1,:),y1(2,:), 'b', y2(1,:),y2(2,:), 'b', y3(1,:),...
y3(2,:), 'b', y4(1,:),y4(2,:), 'b')

```

And here we have the result



1b) We can interpret the matrix norm in terms of these plots as the maximum deformation (in the chosen norm, very important!) that the vectors of the unit circle undergo caused by the matrix A . Be careful with the norm you use to measure the deformed vectors! If you want to determine $\|A\|_1$, for example, the deformed vectors have length $|x| + |y|$, that surely will differ from the Euclidean norm.

2. Inverse Fourier Transform There are two alternatives to solve this exercise.

- **(Changing the code)** Comparing the direct and inverse Fourier transform

$$c_j = \frac{1}{N} \sum_{k=0}^{N-1} y_k \omega_k^{-j} \quad \text{Direct}$$

$$y_k = \sum_{j=0}^{N-1} c_j \omega_k^j \quad \text{Inverse}$$

we observe that the main differences are the minus sign in the exponent of the basis elements and the division by N . We incorporate these changes into the code to obtain this

```
function c=dfft(y)
% c=dfft(y)
% discrete fourier transformation of y
%
c=zeros(size(y));
N=length(y);
omega_N=exp(i*2*pi/N); % Change of sign here!!
%
p=log2(N);
if round(p) ~=p
    error('N is not a power of 2')
end
NRED=N;
for ind=1:p
    NRED_old=NRED;
    NRED=NRED/2;
    NSEG=2^(ind-1); %Number of even/odd segments
    for ISEG=0:NSEG-1
        fac=1;
        for kk=1:NRED;
            k=kk+ISEG*NRED_old;
            alpha_even=y(k)+y(k+NRED);
            alpha_odd =(y(k)-y(k+NRED))*fac;
            fac=fac*omega_N;
            y(k)=alpha_even;
            y(k+NRED)=alpha_odd;
        end;
        omega_N=omega_N^2;
    end;
% Sorting the indices and normalizing by N
% (this could be done by a simple bit-handling instead)
for j=0:N-1,
    jbin=dec2bin(j,p+1);
```

```

jbininv=jbin(p+1:-1:2);
c(j+1)=y(bin2dec(jbininv)+1); % Second Change here!!
end;

```

- **Without changing the code** The goal now is to make the inverse transform without changing the code of `dfmt.m`. The first observation is that the division by N can be done outside of the code. The second problem is how to obtain the changing of sign of the exponent. The solution is very interesting because of the symmetry of the basis elements with respect to the real axis. For this purpose, we take the conjugate of y_k and we observe that

$$y_k = \overline{y_k} = \overline{\sum_{j=0}^{N-1} c_j \omega_k^j} = \sum_{j=0}^{N-1} \overline{c_j \omega_k^j} = \sum_{j=0}^{N-1} \overline{c_j} \omega_k^{-j} = N \left(\frac{1}{N} \sum_{j=0}^{N-1} \overline{c_j} \omega_k^{-j} \right).$$

In conclusion, we just need to take the complex conjugate of the coefficients c_j , pass them through `fft` and multiply by N , and we have reconstructed the samples y_k .

Summarising, we can write a code as short as the following

```

function y=idfft(c)
% idfft without changing dfmt
N=length(c);
y=N*dfmt(conjugate(c));

```

3. Orthogonality

The thesis is equivalent to the following

$$\sum_{j=0}^{N-1} \omega_j^p(t_k) = N \delta_{0p}$$

because of the periodicity of $e^{i\theta}$. Now, if $p = 0$ then

$$\sum_{j=0}^{N-1} \omega_j^0(t_k) = \sum_{j=0}^{N-1} 1 = N$$

Now, let us consider the case were $p \neq 0$. We find now the sum of a geometric series and then

$$\sum_{j=0}^{N-1} \omega_j^p(t_k) = \frac{e^{i \frac{2\pi N p}{N}} - 1}{e^{i \frac{2\pi p}{N}} - 1} = 0$$

which completes the proof.

4. Filtering and high frequency noise cancellation 4a)

The following code filters a signal through the filter $H(Z) = \frac{P(Z)}{Q(Z)}$. The frequencies are fixed by the amount of samples n and we evaluate $H(Z)$ at $Z_k = e^{i\omega_k}$ for $\omega_k = 2\pi \frac{k}{n}$. Here is the code

```
function y=filtering(x,t,P,Q)
n=length(t);
omega=linspace(0,2*pi,n);

z=exp(i*omega);
filt=polyval(P,z)./polyval(Q,z);

xtr=fft(x);
ytr=xtr.*filt;
y=real(ifft(ytr));

figure(1)
stem(t,x)
pause

figure(1)
plot(t,x,'.-',t,y,'.-')
legend('Original','Filtered')
```

4b)

Filtering the signal $x(t)$ with the Butterworth and Chebyshev filters we get the following graphs

5. FFT with powers of 3 (Extra exercise!) This is a very interesting exercise you should solve all by yourself!

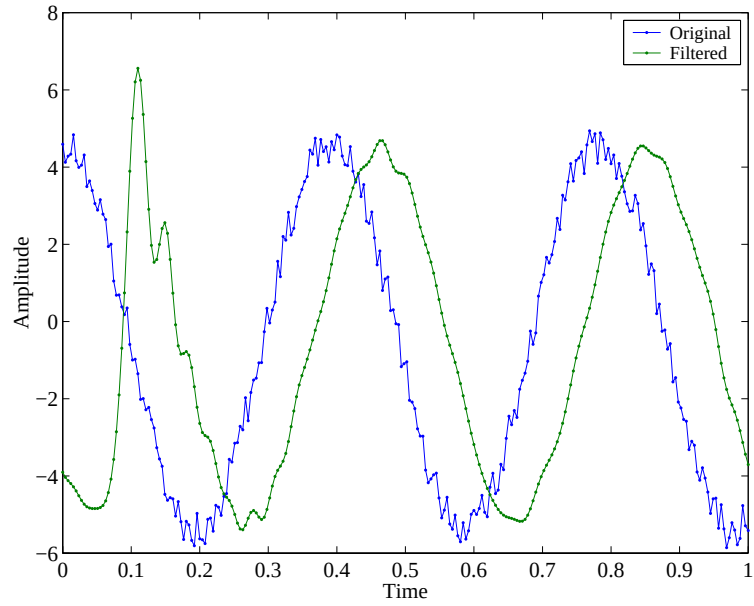


Figure 1: Signal $x(t)$ through Butterworth filter

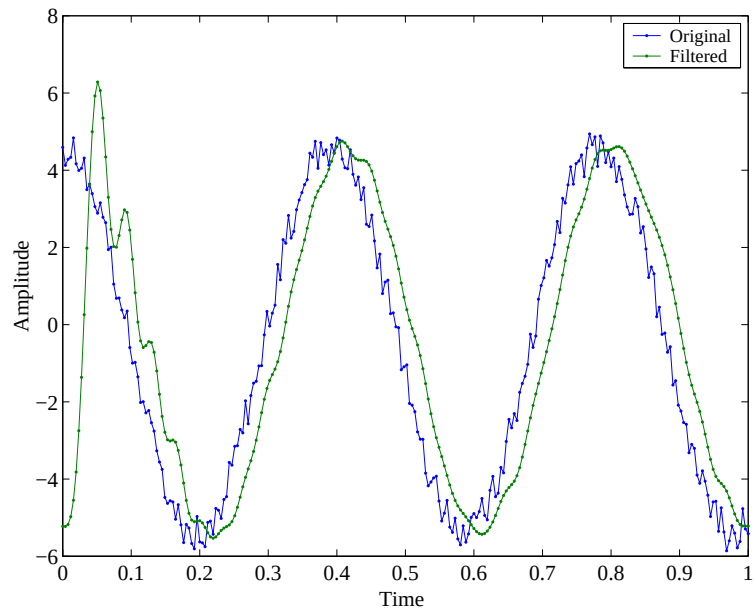


Figure 2: Signal $x(t)$ through Chebyshev filter